ECE 2011
Introduction to Computer Engineering
Spring 2026

Will have some basic
ALU concepts.

Lecture 14, Review
03-03-2026

Linghao Song
Yale ECE

Problem Set 1

- Problem Set 1:
  - March 5 Thursday
  - Select a slot of 75 min at your convenience
  - Boolean Minimization, Finite State Machine, Timing Analysis, Understanding Verilog
  - Prepare your cheat sheet: up to 4 pages (not 4 sheets, pieces of papers), scan or take clear photos of your cheat sheet and solutions
  - Submission no later than 11:59pm ET March 5, submit to Canvas, so you should start working on your pset no later than 10:44pm.
  - No discussion, internet access, textbook, or slides.
  - You may use a non-programmable calculator.

Problem Set 1

- Combinational Logic, Boolean Minimization, Finite State Machine(including sequential logic), Timing Analysis, Understanding Verilog

Binary Values

- Digital abstraction considers **discrete subset** of values
- **Two discrete values:**
  - 1's and 0's
  - 1 = TRUE = HIGH
  - 0 = FALSE = LOW
- How to represent **1 and 0:**
  - voltage levels, rotating gears, fluid levels, etc.
- Digital circuits use **voltage level**s to represent 1 and 0
- ***Bit*: *B*inary dig*it***

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Number Systems

- **Decimal**
  - Base 10
- **Binary**
  - Base 2
- **Hexadecimal**
  - Base 16

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Decimal and Binary Numbers

- **Base 10** (our everyday number system)

1000's column, 100's column, 10's column, 1's column

$$5374_{10} = 5 \times 10^3 + 3 \times 10^2 + 7 \times 10^1 + 4 \times 10^0$$

five thousands, three hundreds, seven tens, four ones

- **Base 2**: Binary numbers

8's column, 4's column, 2's column, 1's column

$$1101_2 = 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 = 13_{10}$$

one eight, one four, no two, one one

Base 2

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Number Conversion

- **Binary to decimal conversion:**
  - Convert 11101_2 to decimal
  - $16\times1 + 8\times1 + 4\times1 + 2\times0 + 1\times1 = 29_{10}$

- **Decimal to binary conversion:**
  - Convert 47_{10} to binary
  - $32\times1 + 16\times0 + 8\times1 + 4\times1 + 2\times1 + 1\times1 = 101111_2$

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Hexadecimal Numbers

Hex Digit	Decimal Equivalent	Binary Equivalent
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
A	10	1010
B	11	1011
C	12	1100
D	13	1101
E	14	1110
F	15	1111

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Two's Complement Numbers

- Most significant bit (msb) has value of -2^{N-1}
- For example, a 4-bit 2's complement number:

1	0	1	1
-2^3	2^2	2^1	2^0

1	0	1	1
-8	4	2	1

Value = -8 + 2 + 1 = -5

(We'll show another way to find this value in a moment.)

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Boolean Variable

- **Boolean variable is a** variable whose value is only one of two possible values:
  - TRUE or FALSE
  - Equivalently, 1 or 0
  - The variable could be A, B, X, Y …
- **Boolean expression** is an expression using Boolean variables.
  - E.g., Y = A + B

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Logic Gates

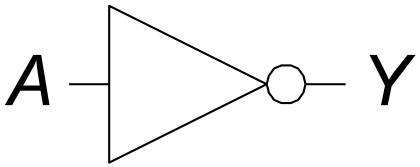
- **Perform logic functions:**
  - inversion (NOT), AND, OR, NAND, NOR, etc.
- **Single-input:**
  - NOT gate, buffer
- **Two-input:**
  - AND, OR, XOR, NAND, NOR, XNOR
- **Multiple-input**

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Single-Input Logic Gates

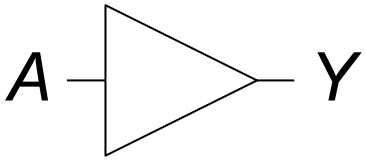
NOT



$Y = \overline{A}$

A	Y
0	1
1	0

BUF



$Y = A$

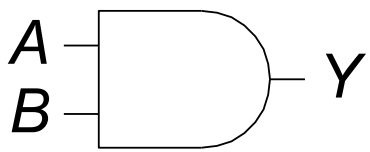
A	Y
0	0
1	1

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Two-Input Logic Gates

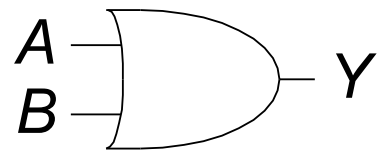
AND



$Y = AB$

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

OR



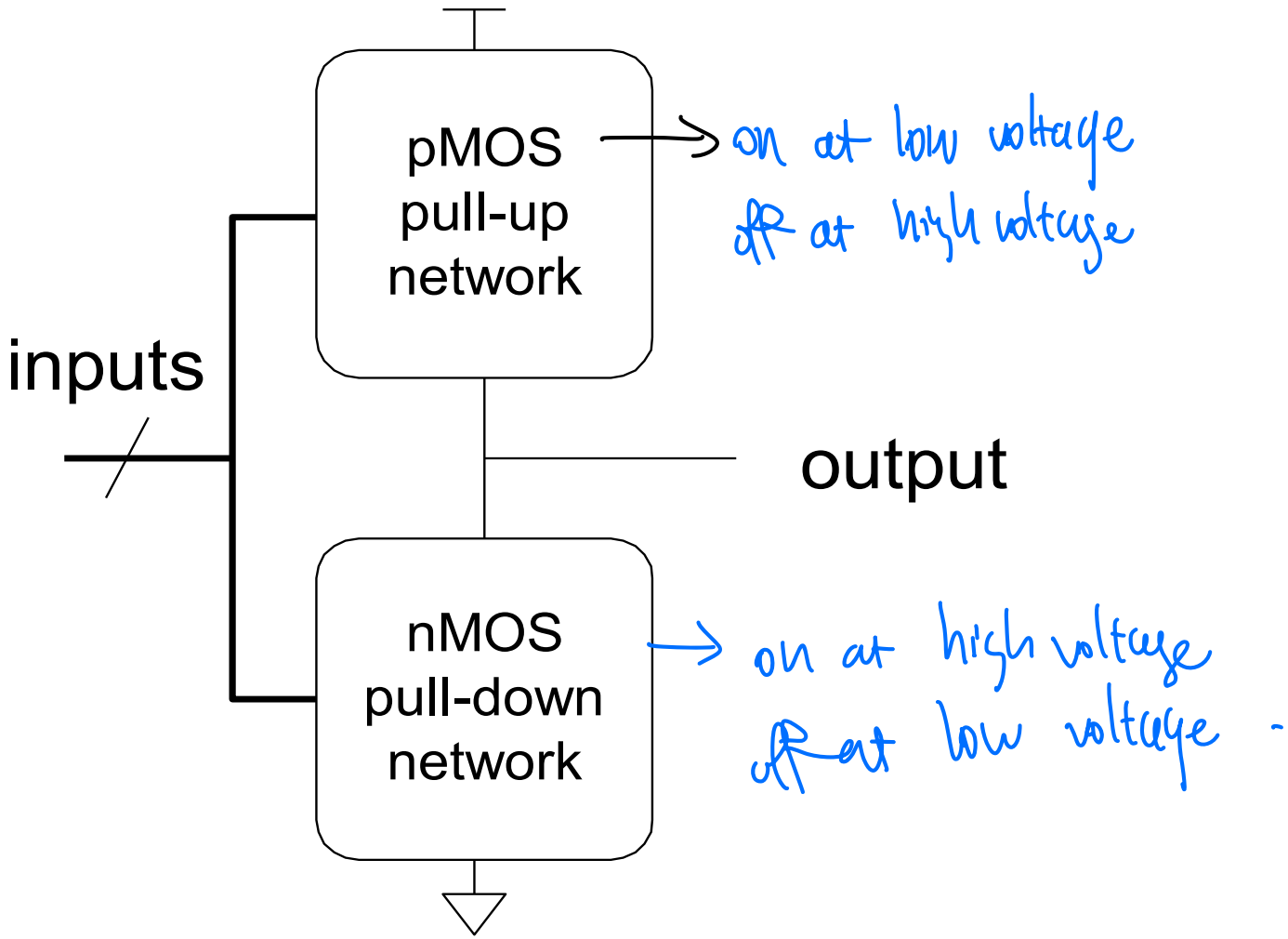
$Y = A + B$

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

CMOS Gate Structure

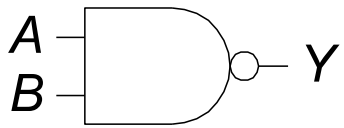


© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

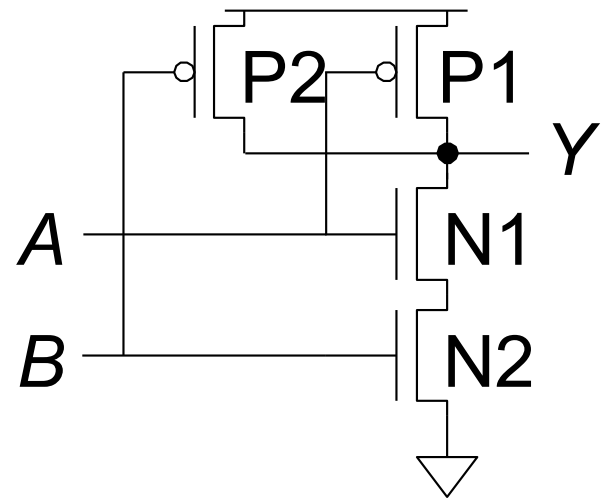
CMOS Gates: NAND Gate

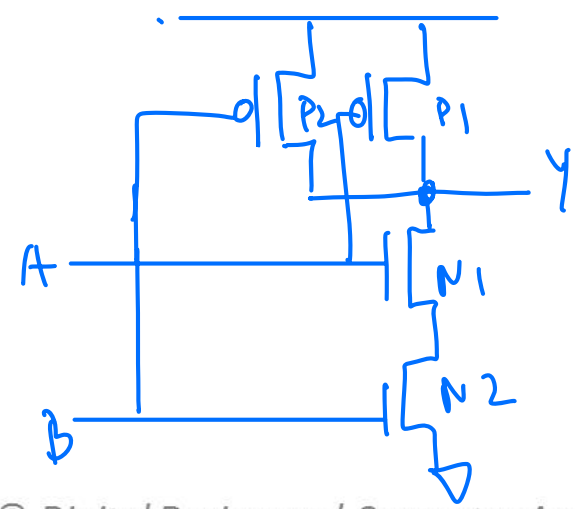
NAND



$Y = \overline{AB}$

A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0





A	*B*	P1	P2	N1	N2	*Y*
0	0	ON	ON	OFF	OFF	1
0	1	ON	OFF	OFF	ON	1
1	0	OFF	ON	ON	OFF	1
1	1	OFF	OFF	ON	ON	0

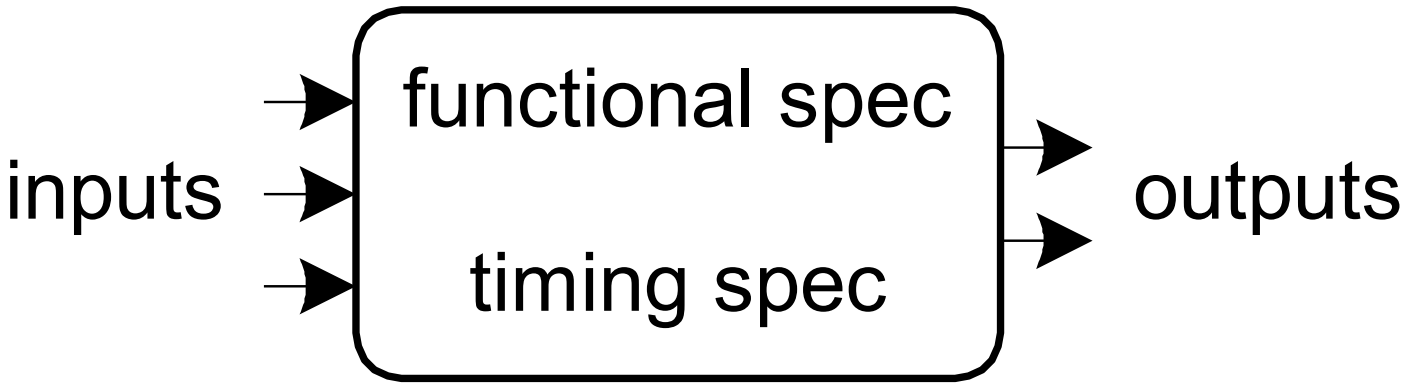
© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Logic Circuits

A logic circuit is composed of:

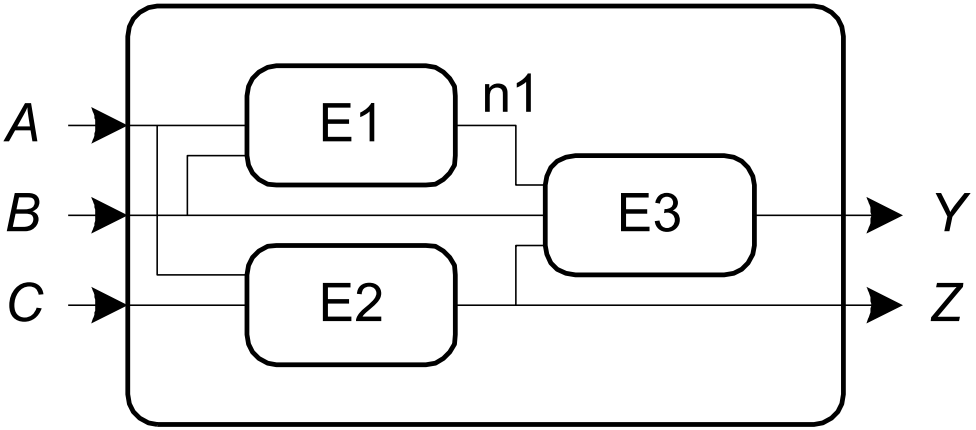
- Inputs
- Outputs
- Functional specification
- Timing specification



© *Digital Design and Computer Architecture*, 2nd Edition, 2012

Circuits

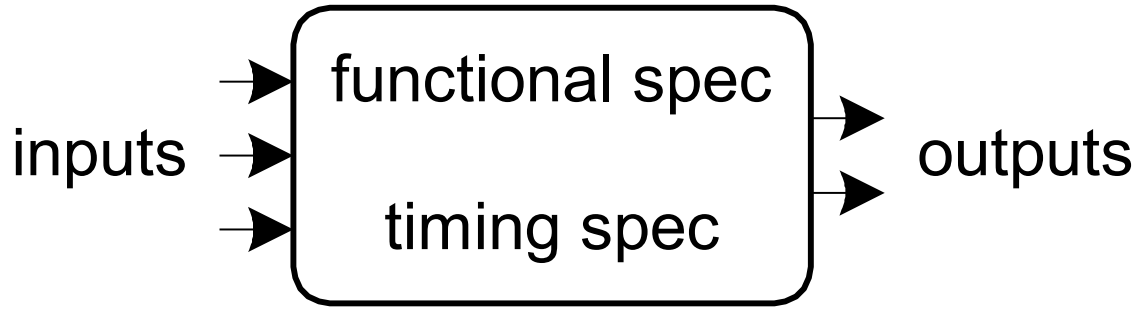
- **Nodes**
  - Inputs: *A*, *B*, *C*
  - Outputs: *Y*, *Z*
  - Internal: n1
- **Circuit elements**
  - E1, E2, E3
  - Each a circuit



© *Digital Design and Computer Architecture*, 2nd Edition, 2012

Types of Logic Circuits

- **Combinational Logic**
  - Memoryless
  - Outputs determined by current values of inputs
- **Sequential Logic**
  - Has memory
  - Outputs determined by previous and current values of inputs



© *Digital Design and Computer Architecture*, 2nd Edition, 2012

Some Definitions

- **Complement:** variable with a bar over it
  $\bar{A}, \bar{B}, \bar{C}$
- **Literal:** variable or its complement
  $A, \bar{A}, B, \bar{B}, C, \bar{C}$
- **Implicant:** product of literals
  $AB\bar{C}, \bar{A}C, BC$
- **Minterm:** product that includes **all** input variables
  $AB\bar{C}, A\bar{B}\bar{C}, ABC$
- **Maxterm:** sum that includes **all** input variables
  $(A+\bar{B}+C), (\bar{A}+B+\bar{C}), (\bar{A}+\bar{B}+C)$

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Sum-of-Products (SOP) v.s. Product-of-Sums (POS)

- All Boolean equations can be written in SOP form
- Each row has a **minterm**
- A minterm is a **product** (AND) of literals
- Each minterm is **TRUE** for that row (and only that row)
- Form function by **ORing minterms** where **output is 1**
- Thus, a **sum** (OR) of **products** (AND terms)
- How to list all minterms?
  - A product of literals of all variables for 1
- How to find the minterms in SOP and form SOP?
  - **ORing minterms** where **output/Y is 1**

- All Boolean equations can be written in POS form
- Each row has a **maxterm**
- A maxterm is a **sum** (OR) of literals
- Each maxterm is **FALSE** for that row (and only that row)
- Form function by **ANDing maxterms** where **output is 0**
- Thus, a **product** (AND) of **sums** (OR terms)
- How to list all maxterms?
  - A sum of literals of all variables for 0
- How to find the maxterms in POS and form POS?
  - **ANDing maxterms** where **output/Y is 0**

$Y = \bar{A}B + AB = (\bar{A}\cdot B)\cdot(\bar{A}\cdot B)$

$= (A+\bar{B})(\bar{A}+\bar{B})$

A	*B*	minterm	minterm name	*Y*	maxterm	maxterm name
0	0	$\bar{A}\bar{B}$	m_0	0	$A+B$	M_0
0	1	$\bar{A}B$	m_1	1	$A+\bar{B}$	M_1
1	0	$A\bar{B}$	m_2	0	$\bar{A}+B$	M_2
1	1	AB	m_3	1	$\bar{A}+\bar{B}$	M_3

Here Y should be 0.

$$Y = \mathrm{F}(A, B) = \bar{A}B + AB$$

$$Y = \mathrm{F}(A, B) = (A+B)\bullet(\bar{A}+B)$$

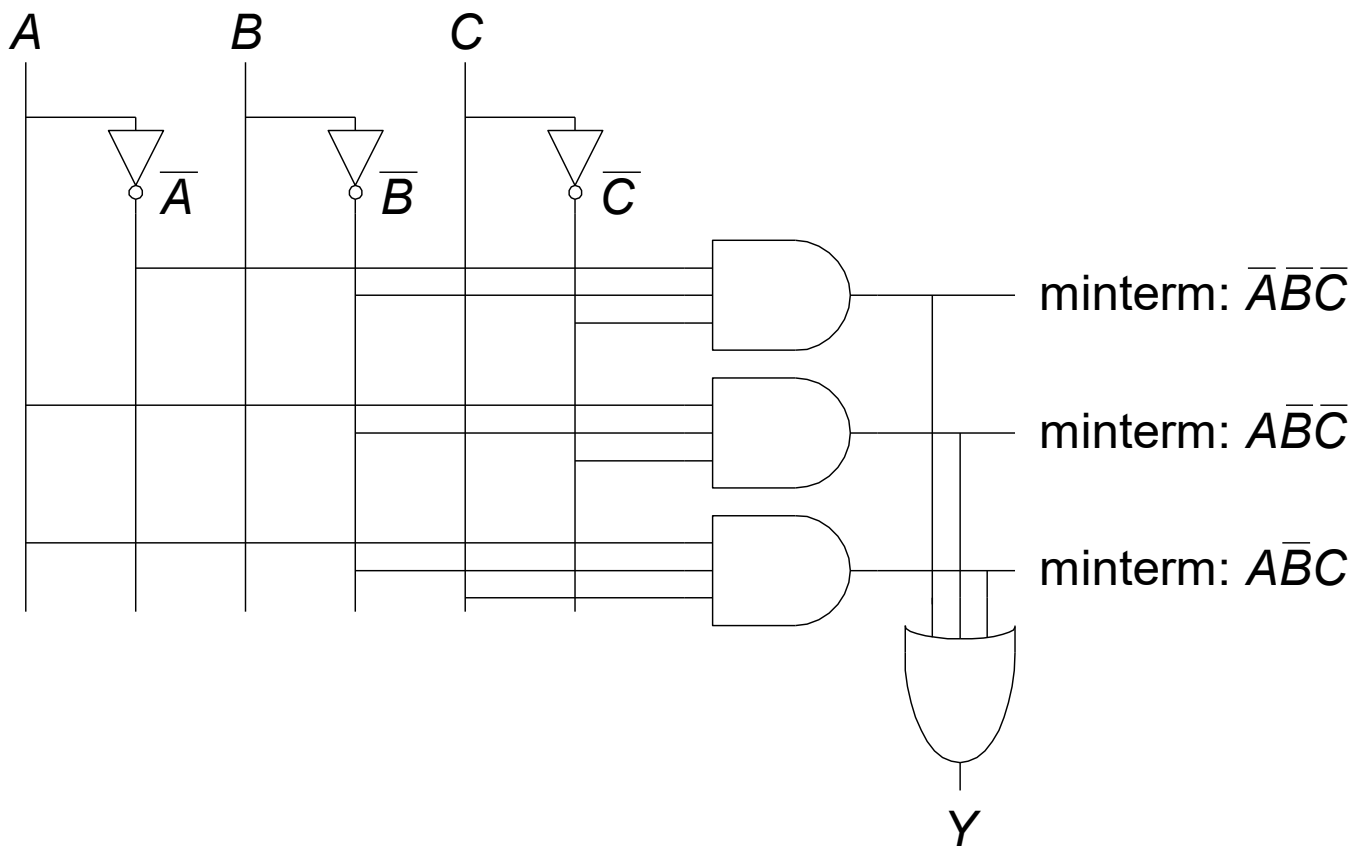
- SOP, POS, Function, are equivalent. F = SOP = POS.
- Each minterm are maxterm in the same row the negation of each other. E.g., NOT($\bar{A}B$)=$A+\bar{B}$.

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

From Logic to Gates

- Two-level logic: ANDs followed by ORs
- Example: $Y = \overline{A}\,\overline{B}\,\overline{C} + A\overline{B}\,\overline{C} + A\overline{B}C$



© *Digital Design and Computer Architecture*, 2nd Edition, 2012

Karnaugh Maps (K-Maps)

- Boolean expressions can be minimized by combining terms
- K-maps minimize equations graphically

A	B	C	Y
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	0

00 01 11 10

00
01
11
10

Y: C \ AB	00	01	11	10
0	1	0	0	0
1	1	0	0	0

Y: C \ AB	00	01	11	10
0	$\bar{A}\bar{B}\bar{C}$	$\bar{A}B\bar{C}$	$AB\bar{C}$	$A\bar{B}\bar{C}$
1	$\bar{A}\bar{B}C$	$\bar{A}BC$	ABC	$A\bar{B}C$

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

K-Map Rules

- Adjacent rows/cols only have one bit flip
- **Every 1 must be circled** at least once
- Each circle must span a **power of 2** (i.e. 1, 2, 4) blocks in each direction
- Each circle must be as **large** as possible
- A circle may **wrap around the edges**
- Circle a "**don't care**" (X) **only if it helps** minimize the equation

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

4-Input K-Map

A	B	C	D	Y
0	0	0	0	1
0	0	0	1	0
0	0	1	0	1
0	0	1	1	1
0	1	0	0	0
0	1	0	1	1
0	1	1	0	1
0	1	1	1	1
1	0	0	0	1
1	0	0	1	1
1	0	1	0	1
1	0	1	1	0
1	1	0	0	0
1	1	0	1	0
1	1	1	0	0
1	1	1	1	0

Y: CD \ AB	00	01	11	10
00	1	0	0	1
01	0	1	0	1
11	1	1	0	0
10	1	1	0	1

$$Y = \overline{A}C + \overline{A}BD + A\overline{B}\,\overline{C} + \overline{B}\,\overline{D}$$

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

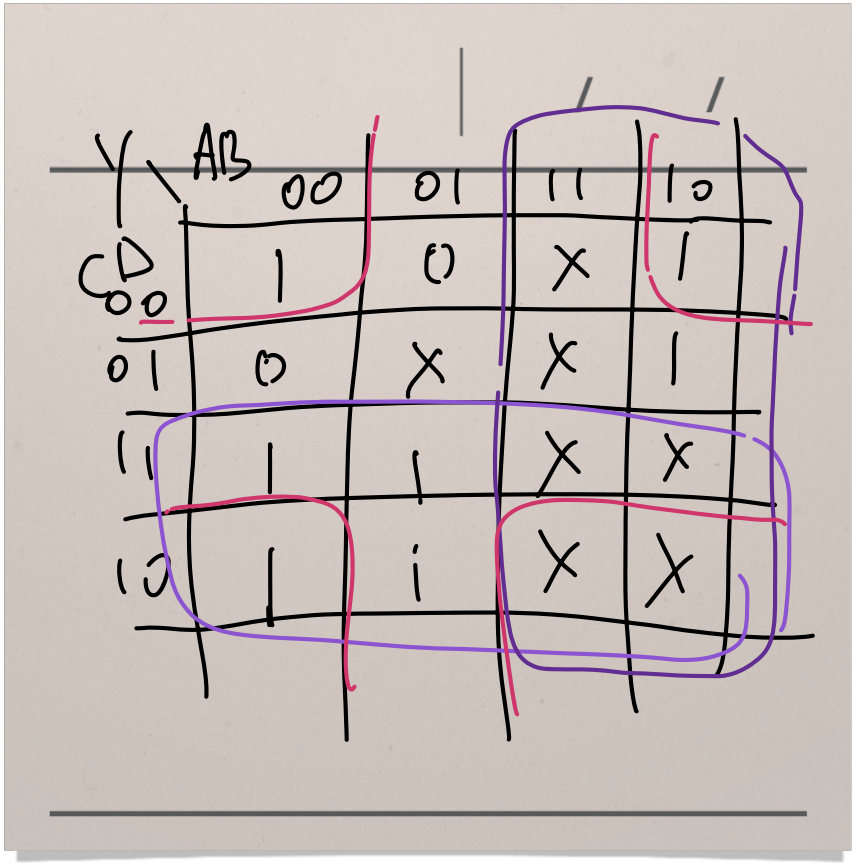
K-Maps with Don't Cares

Can you construct a K-map quickly

If you have Don't Care in inputs ↓ enumerate 0 & 1

A	B	C	D	Y
0	0	0	0	1
0	0	0	1	0
0	0	1	0	1
0	0	1	1	1
0	1	0	0	0
0	1	0	1	X
0	1	1	0	1
0	1	1	1	1
1	0	0	0	1
1	0	0	1	1
1	0	1	0	X
1	0	1	1	X
1	1	0	0	X
1	1	0	1	X
1	1	1	0	X
1	1	1	1	X

① C
② A
③ $\bar{B}\bar{D}$



$Y = A + \bar{B}\bar{D} + C$

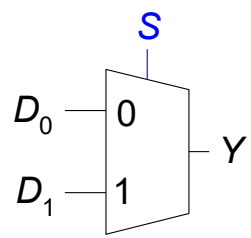
↳ Here $Y = A + \bar{B}\bar{D} + C$.

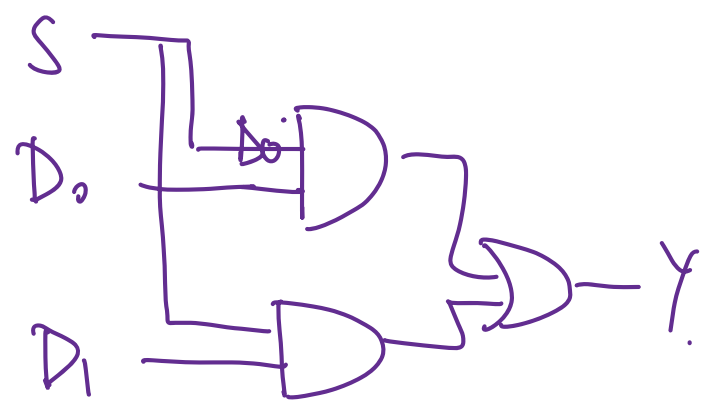
© Digital Design and Computer Architecture, 2nd Edition, 2012

ELSEVIER

Multiplexer (Mux)

- Selects between one of *N* inputs to connect to output
- $\log_2 N$-bit select input – control input
- **Example:** **2:1 Mux**





S	D_1	D_0	Y
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

S	Y
0	D_0
1	D_1

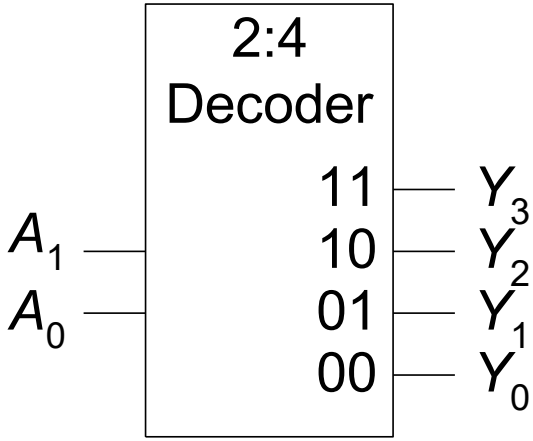
chose a signal.

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Decoders

- N inputs, 2^N outputs
- **One-hot** outputs: only **one output HIGH** at once



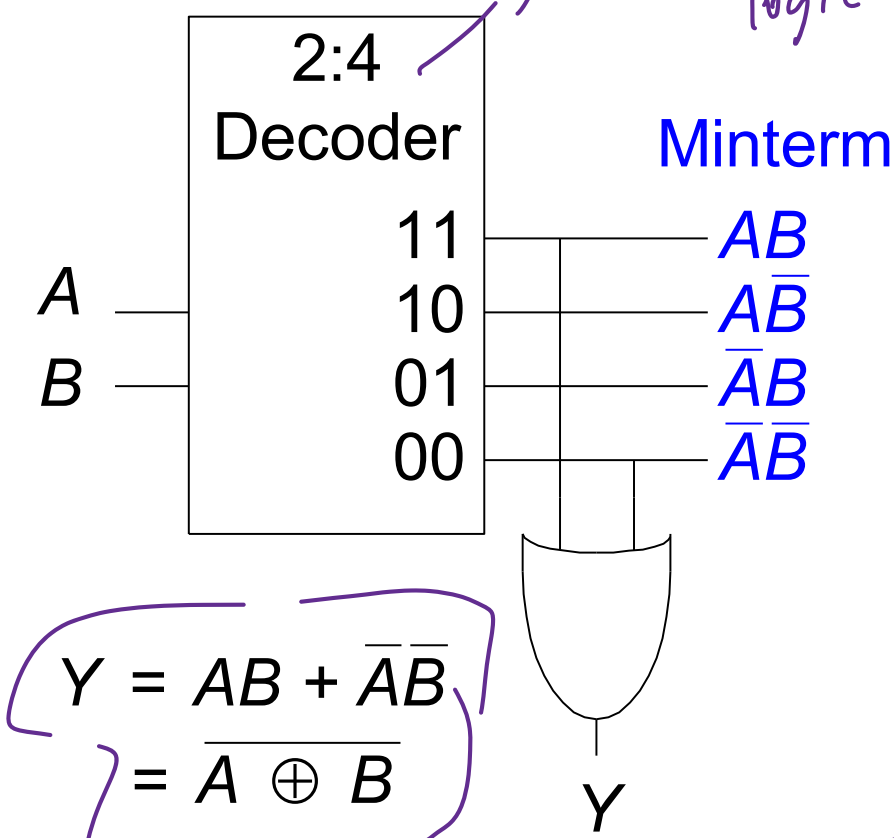
A_1	A_0	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	1
0	1	0	0	1	0
1	0	0	1	0	0
1	1	1	0	0	0

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

Logic Using Decoders

OR the minterms

can use to perform logic functions.



given 2 inputs there are all combos of these 2 inputs

$$Y = AB + \overline{A}\overline{B}$$
$$= \overline{A \oplus B}$$

this is easily confused with $AB + \overline{AB}$.

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

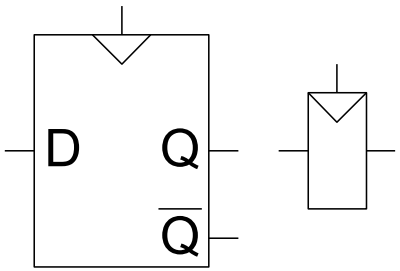
ELSEVIER

D Flip-Flop

Memory unit in sequential logic.

- **Inputs:** *CLK*, *D*
- **Function**
  - Samples *D* on rising edge of *CLK*
    - When *CLK* rises from 0 to 1, *D* passes through to *Q*
    - Otherwise, *Q* holds its previous value
  - *Q* changes only on rising edge of *CLK*
- Called *edge-triggered*
- Activated on the clock edge

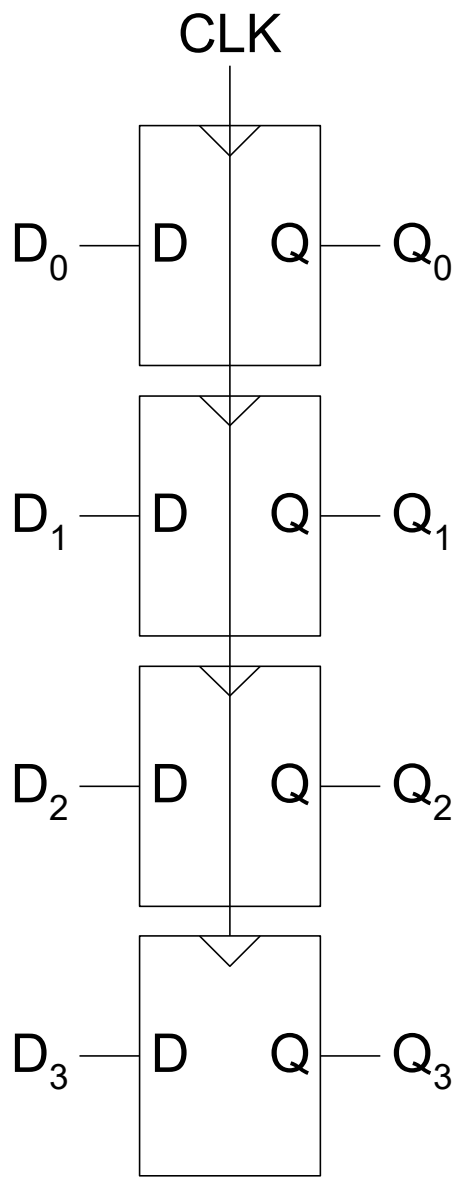
D Flip-Flop Symbols



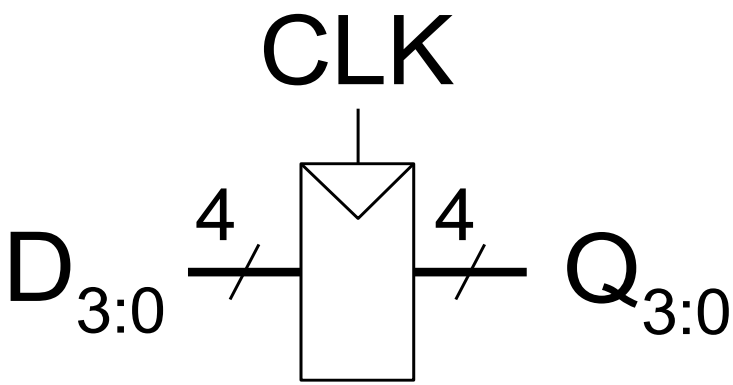
© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Registers



- Registers are multi-bit (N-bit) flip-flops with a common clock

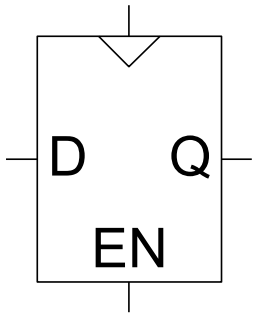


© *Digital Design and Computer Architecture*, 2nd Edition, 2012

Enabled Flip-Flops

- **Inputs:** *CLK*, *D*, *EN*
  - The enable input (*EN*) controls when new data (*D*) is stored
- **Function**
  - ***EN* = 1:** *D* passes through to *Q* on the clock edge
  - ***EN* = 0:** the flip-flop retains its previous state

Symbol



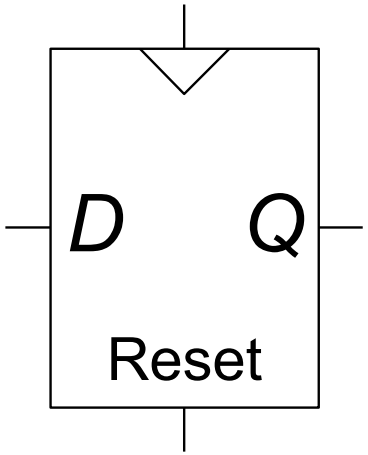
© *Digital Design and Computer Architecture*, 2nd Edition, 2012

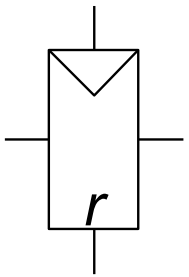
ELSEVIER

Resettable Flip-Flops

- **Inputs:** *CLK*, *D*, *Reset*
- **Function:**
  - ***Reset* = 1:** *Q* is forced to 0
  - ***Reset* = 0:** flip-flop behaves as ordinary D flip-flop

Symbols

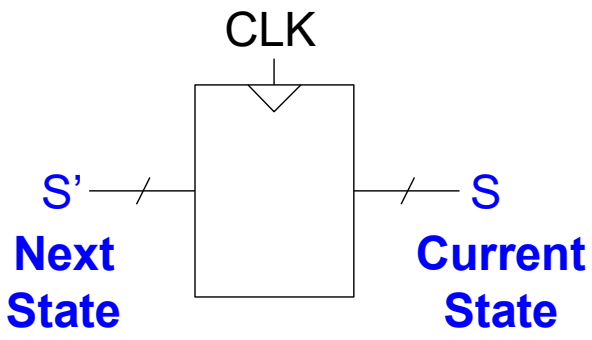


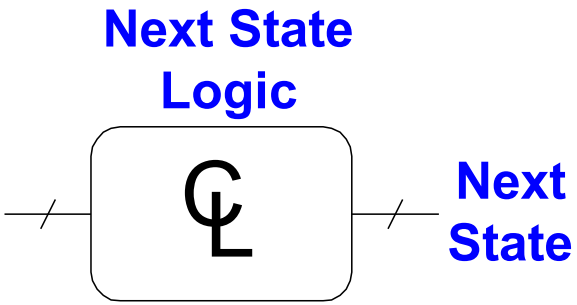


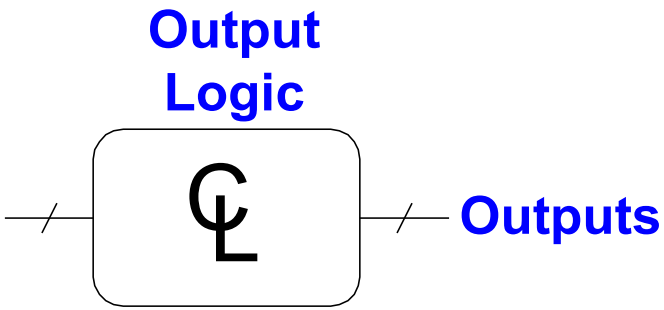
© *Digital Design and Computer Architecture*, 2nd Edition, 2012

Finite State Machine (FSM)

- Consists of:
  - **State register**
    - Stores current state
    - Loads next state at clock edge
  - **Combinational logic**
    - Computes the next state
    - Computes the outputs







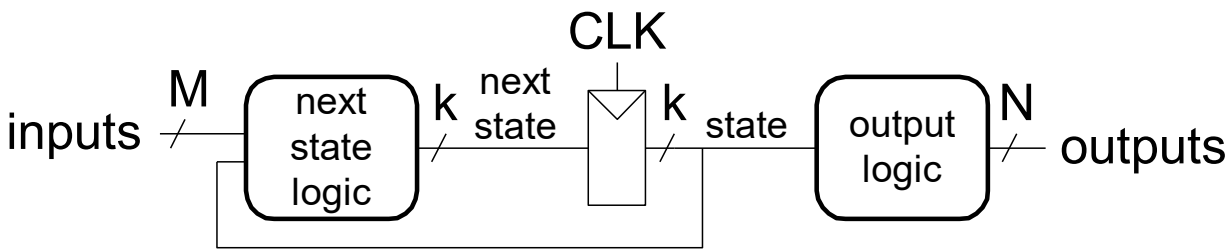
© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

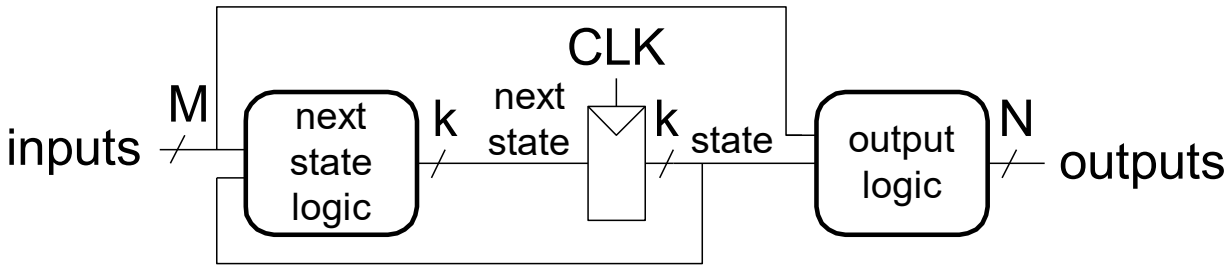
Finite State Machines (FSMs)

- Next state determined by current state and inputs
- Two types of finite state machines differ in output logic:
  - **Moore FSM:** outputs depend only on current state
  - **Mealy FSM:** outputs depend on current state *and* inputs

Moore FSM



Mealy FSM



© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

FSM Design Procedure

1. Identify **inputs** and **outputs**
2. Sketch **state transition diagram**
3. Write **state transition table**
4. Select **state(input, output) encodings**
5. For Moore machine:
   1. Rewrite state transition table with state **encodings**
   2. Write **output table**
6. For a Mealy machine:

   Rewrite combined state transition and output table with state **encodings**
7. Write **Boolean equations** for next state and output logic
8. Sketch the circuit **schematic**

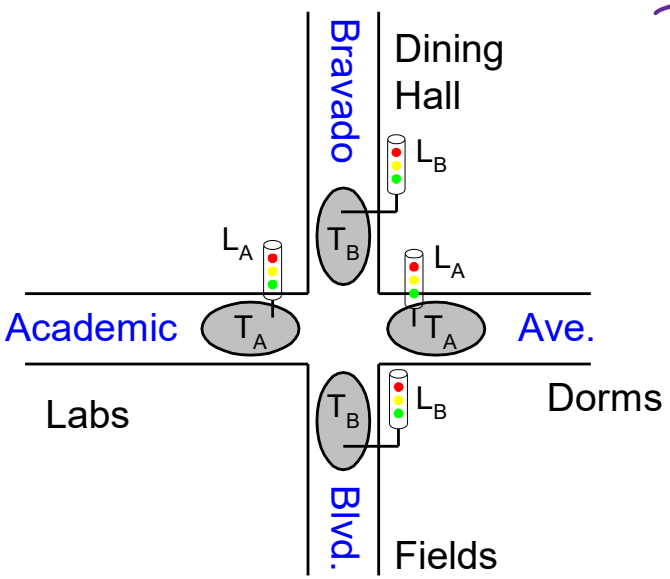
© *Digital Design and Computer Architecture*, 2nd Edition, 2012

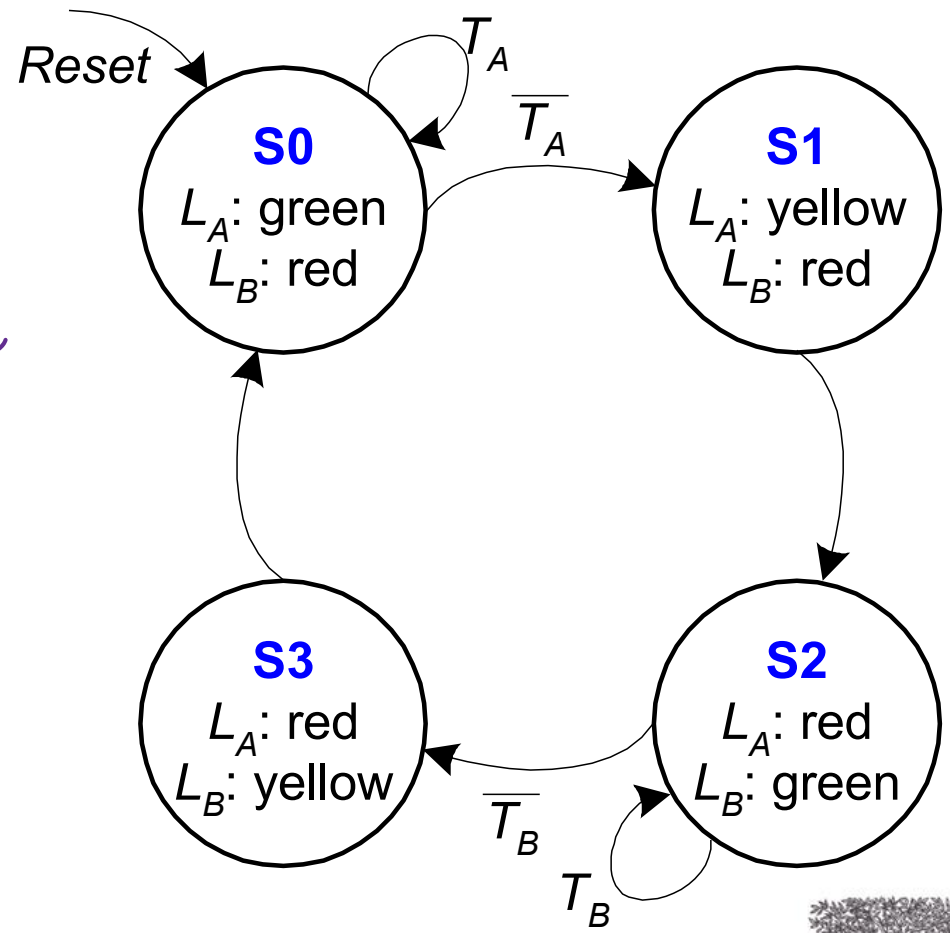
ELSEVIER

FSM State Transition Diagram

- **Moore FSM:** outputs labeled in each state
- **States:** Circles
- **Transitions:** Arcs

go over this example again!





© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

FSM Encoded State Transition Table

Current State		Inputs		Next State	
S_1	S_0	T_A	T_B	S'_1	S'_0
0	0	0	X	0	1
0	0	1	X	0	0
0	1	X	X	1	0
1	0	X	0	1	1
1	0	X	1	1	0
1	1	X	X	0	0

State	Encoding
S0	00
S1	01
S2	10
S3	11

$$S'_1 = S_1 \oplus S_0$$

$$S'_0 = \overline{S_1}\,\overline{S_0}\,\overline{T_A} + S_1\overline{S_0}\,\overline{T_B}$$

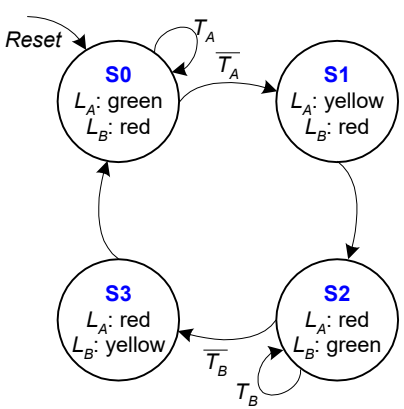
© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

FSM Output Table

Current State		Outputs			
S_1	S_0	L_{A1}	L_{A0}	L_{B1}	L_{B0}
0	0	0	0	1	0
0	1	0	1	1	0
1	0	1	0	0	0
1	1	1	0	0	1

Output	Encoding
green	00
yellow	01
red	10

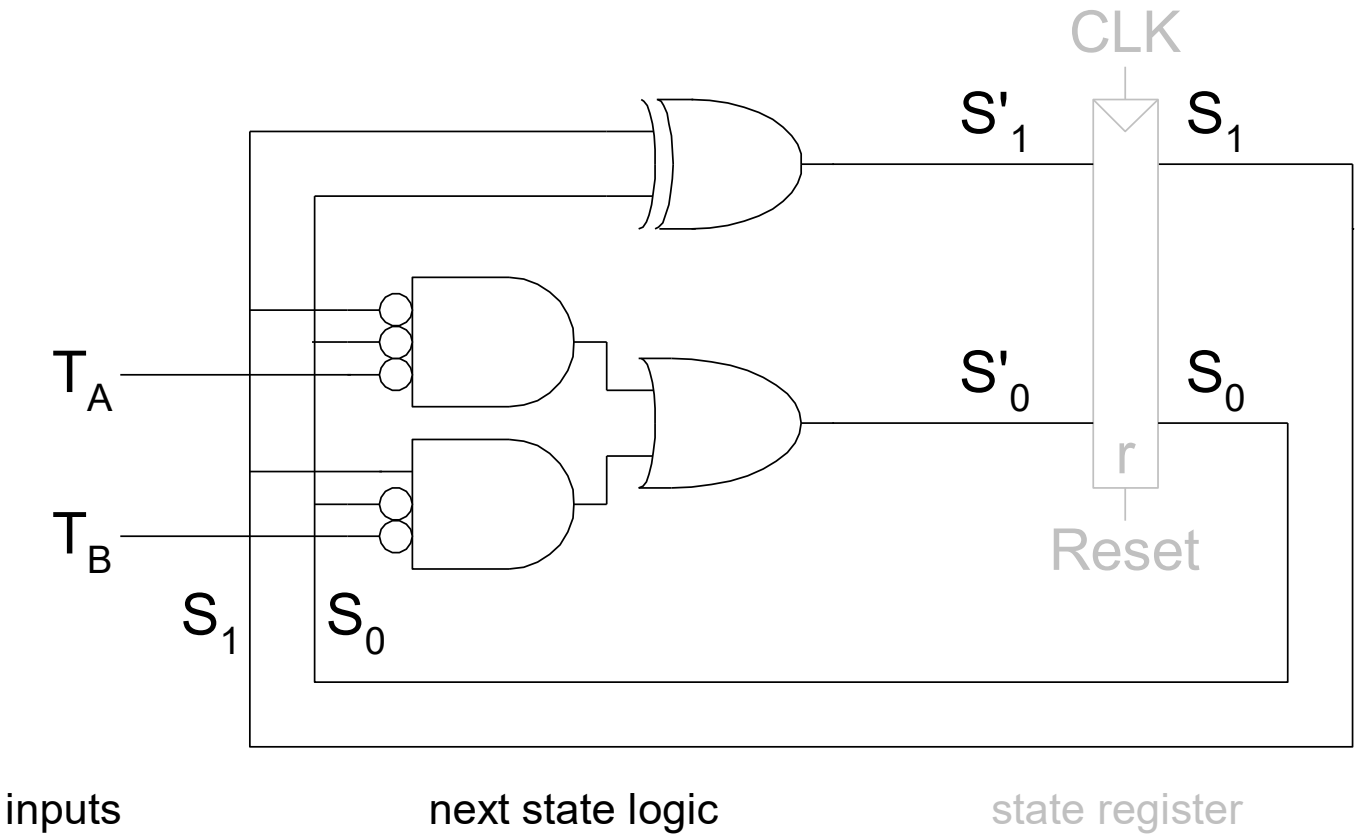


$$L_{A1} = S_1$$
$$L_{A0} = \overline{S_1} S_0$$
$$L_{B1} = \overline{S_1}$$
$$L_{B0} = S_1 S_0$$

© Digital Design and Computer Architecture, 2nd Edition, 2012

ELSEVIER

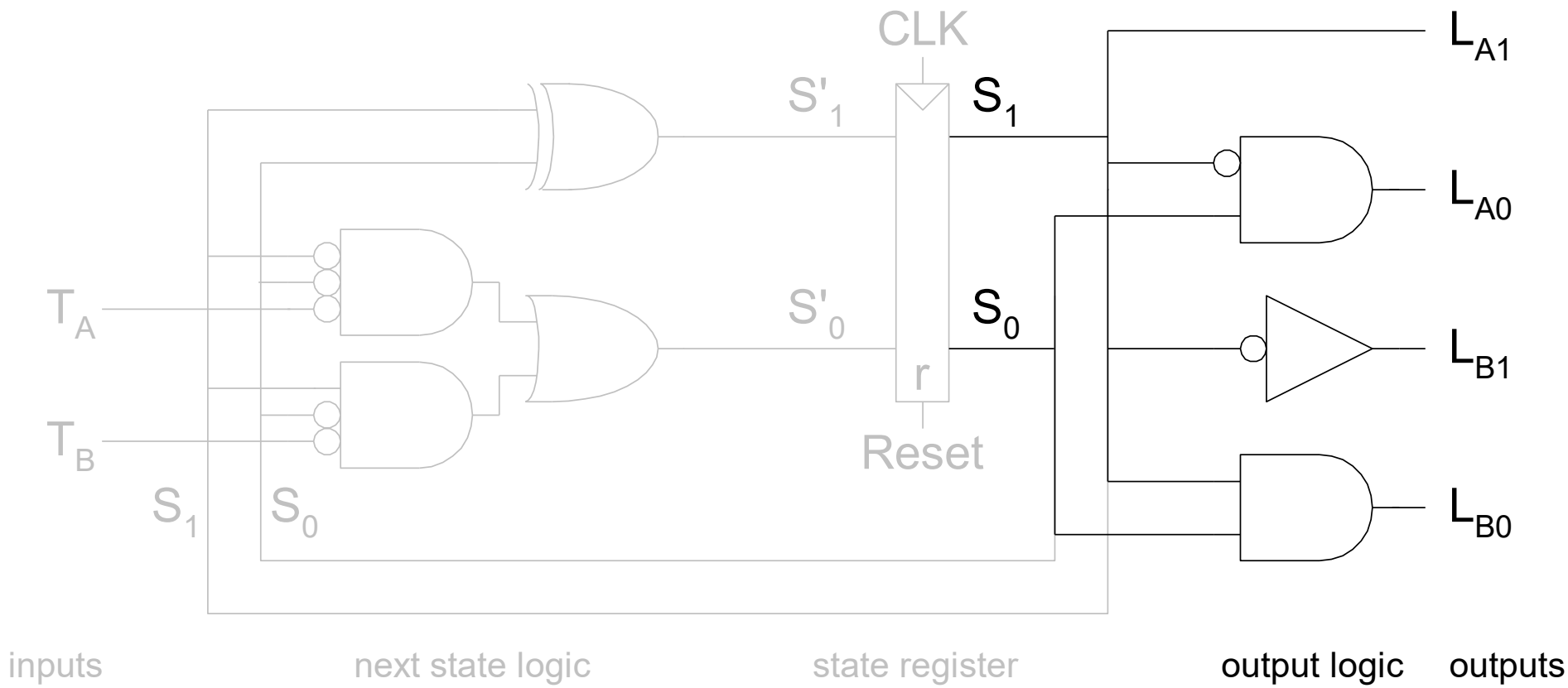
FSM Schematic: Next State Logic



© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

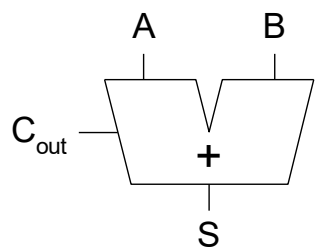
FSM Schematic: Output Logic



© *Digital Design and Computer Architecture*, 2nd Edition, 2012

1-Bit Adders

Half Adder

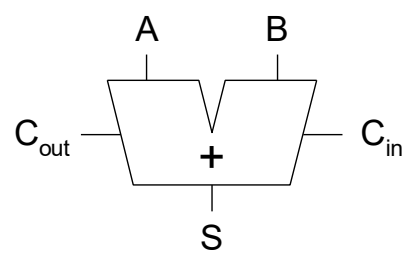


A	B	C_{out}	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

$S = A \oplus B$

$C_{out} = AB$

Full Adder



C_{in}	A	B	C_{out}	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

$S = A \oplus B \oplus C_{in}$

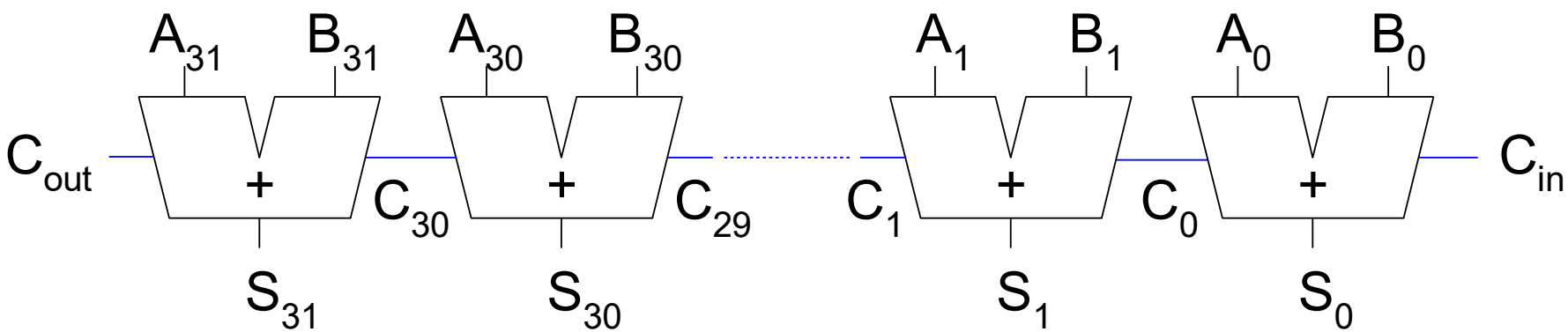
$C_{out} = AB + AC_{in} + BC_{in}$

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

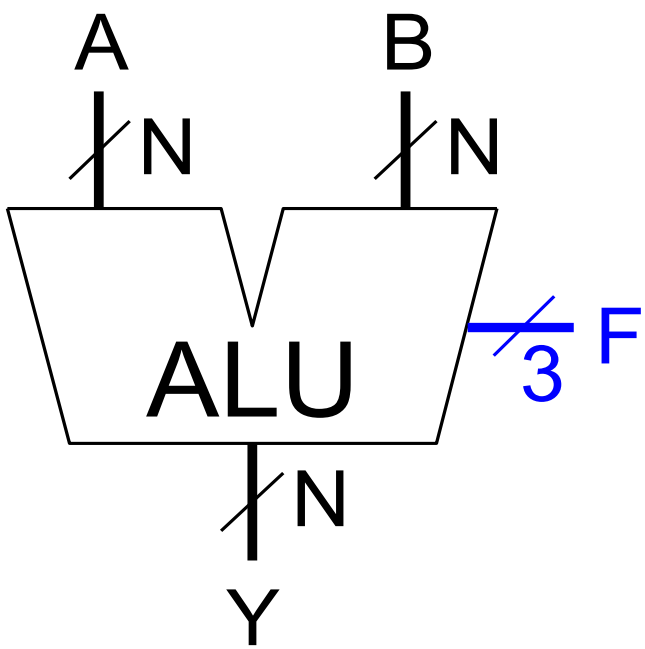
Ripple-Carry Adder

- Chain 1-bit adders together
- Carry ripples through entire chain
- Disadvantage: **slow**



© *Digital Design and Computer Architecture*, 2nd Edition, 2012

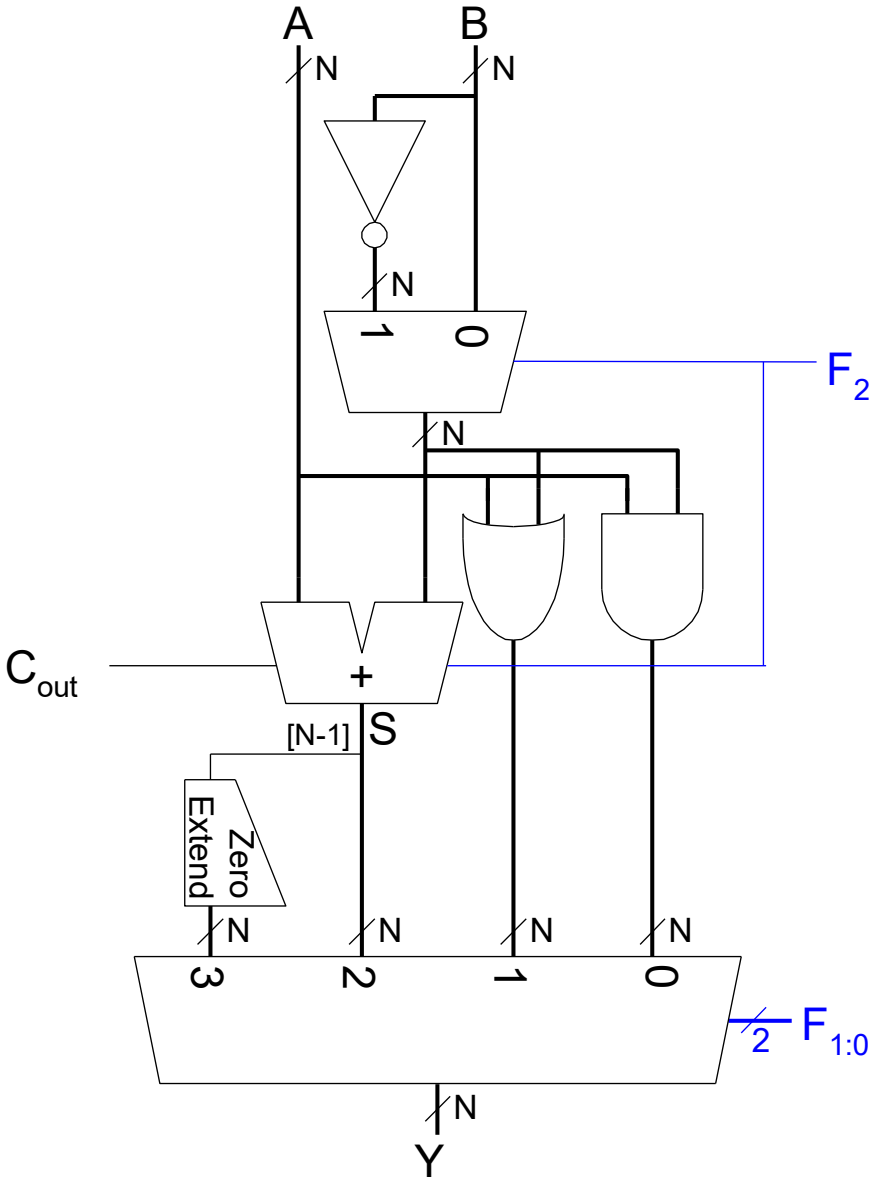
Arithmetic Logic Unit (ALU)



$F_{2:0}$	Function
000	A & B
001	A | B
010	A + B
011	not used
100	A & ~B
101	A | ~B
110	A - B
111	SLT

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ALU Design

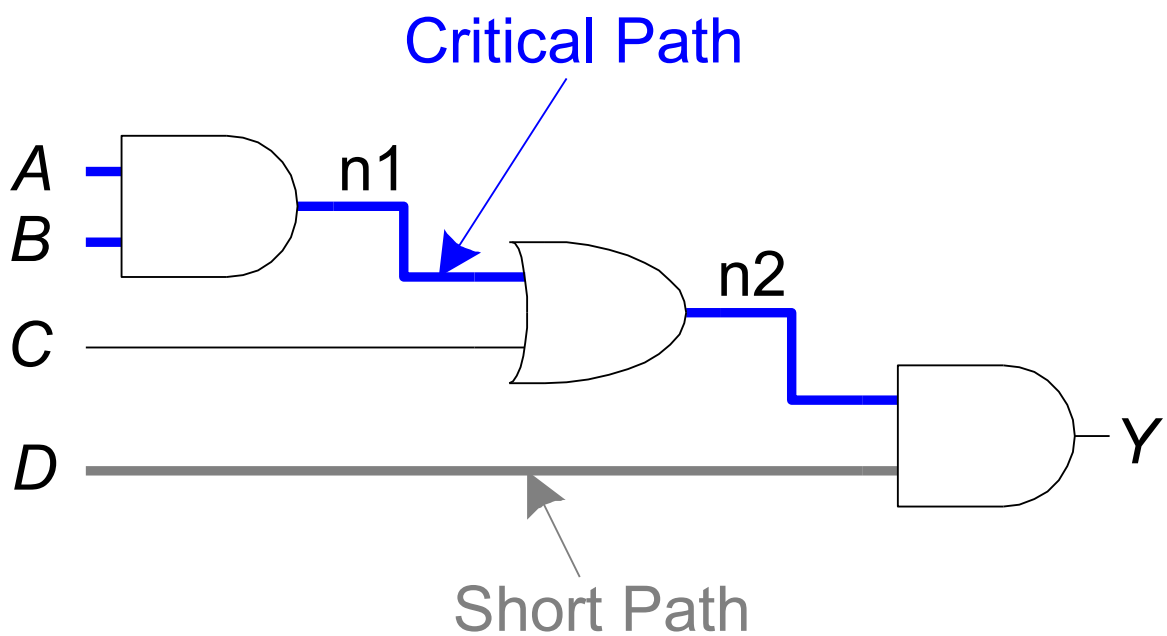


$F_{2:0}$	Function
000	A & B
001	A | B
010	A + B
011	not used
100	A & ~B
101	A | ~B
110	A - B
111	SLT

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Critical (Long) & Short Paths



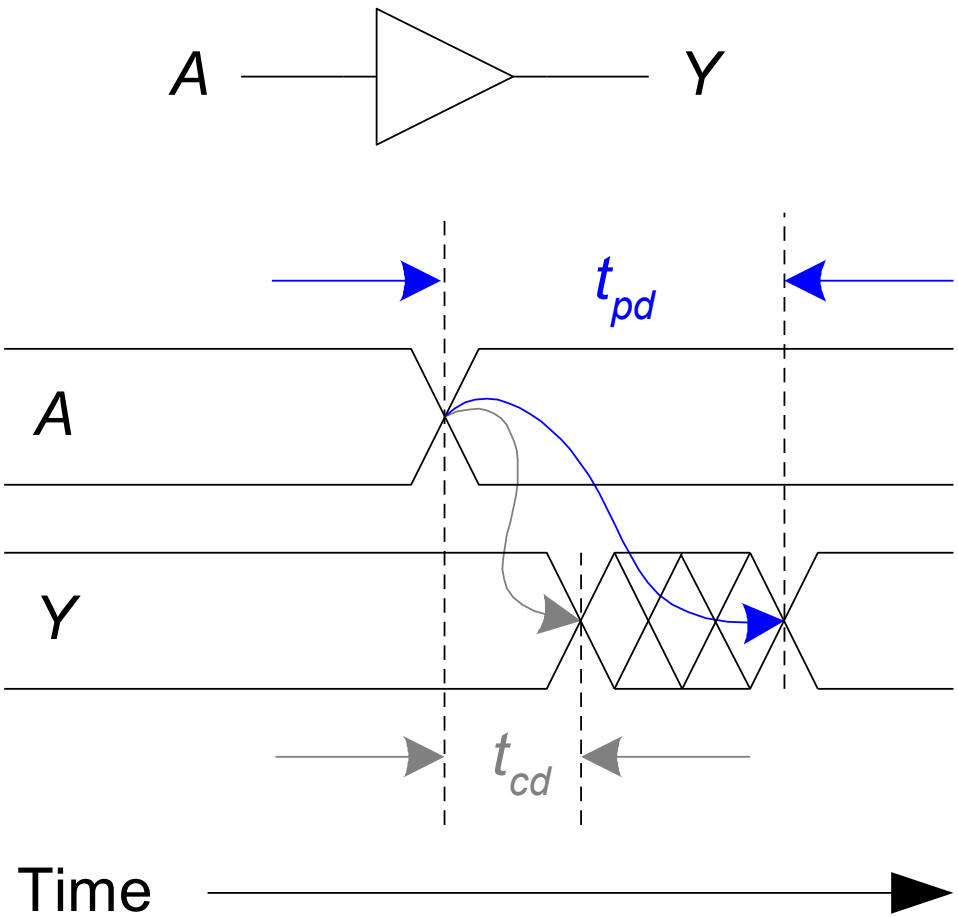
Critical (Long) Path: $t_{pd} = 2t_{pd_AND} + t_{pd_OR}$

Short Path: $t_{cd} = t_{cd_AND}$

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

Propagation & Contamination Delay

- **Propagation delay:** t_{pd} = max delay from input to output
- **Contamination delay:** t_{cd} = min delay from input to output

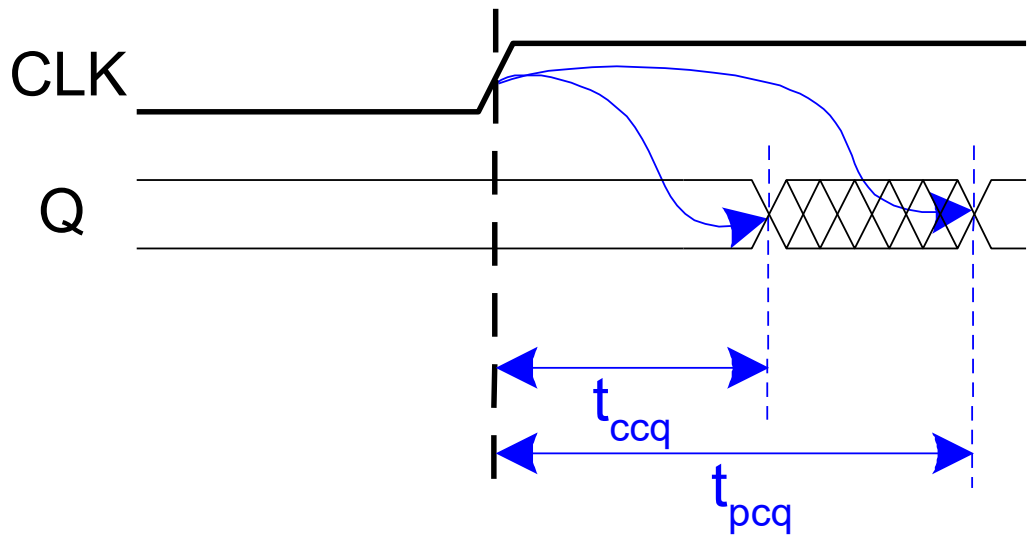


© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Output Timing Constraints

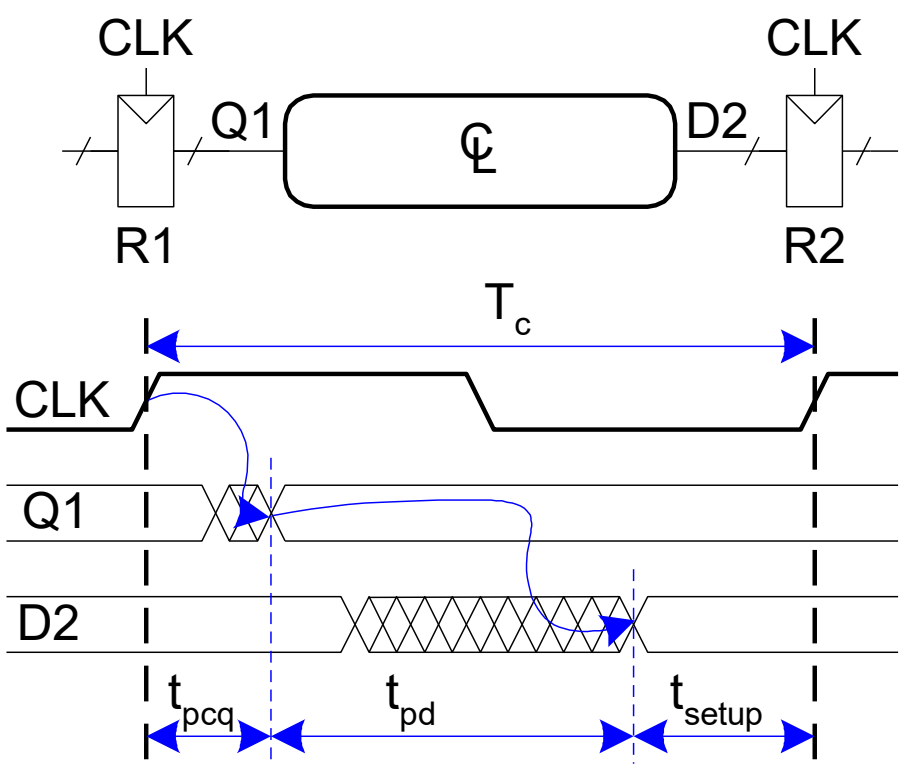
- **Propagation delay:** t_{pcq} = time after clock edge that the output *Q* is guaranteed to be stable (i.e., to stop changing)
- **Contamination delay:** t_{ccq} = time after clock edge that *Q* might be unstable (i.e., start changing)



© *Digital Design and Computer Architecture*, 2nd Edition, 2012

Setup Time Constraint

- Depends on the **maximum** delay from register R1 through the combinational logic to R2
- The input to register R2 must be stable at least t_{setup} before clock edge



determined by the properties of the register.

$$T_c \geq t_{pcq} + t_{pd} + t_{setup}$$

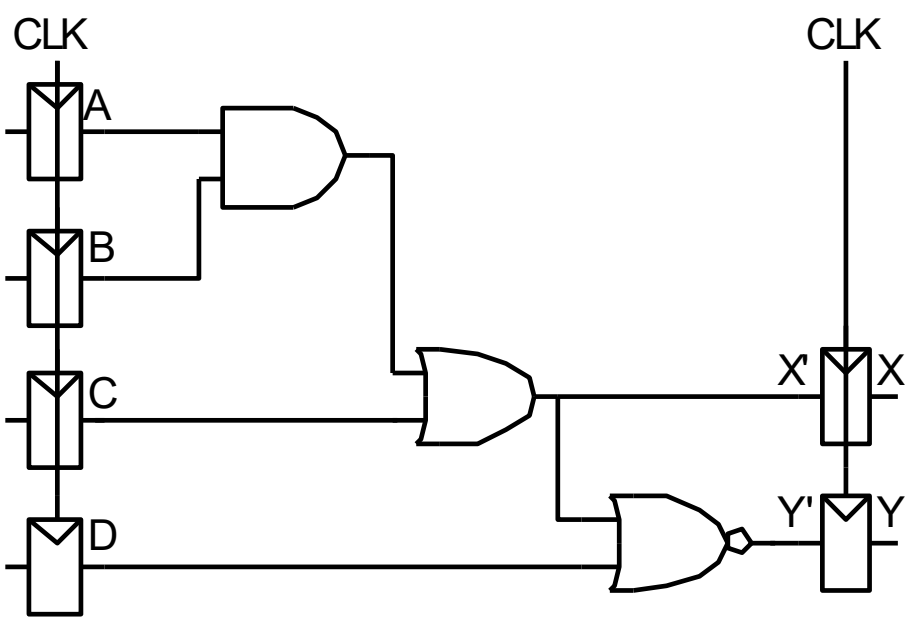
$$t_{pd} \leq T_c - (t_{pcq} + t_{setup})$$

$(t_{pcq} + t_{setup})$: *sequencing overhead*

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Timing Analysis



Timing Characteristics

t_{ccq} = 30 ps

t_{pcq} = 50 ps

t_{setup} = 60 ps

t_{hold} = 70 ps

per gate

t_{pd} = 35 ps

t_{cd} = 25 ps

how do I understand/interpret thold.

t_{pd} = 3 x 35 ps = 105 ps

t_{cd} = 25 ps

Setup time constraint:

$T_c \geq (50 + 105 + 60)$ ps = 215 ps

$f_c = 1/T_c$ = 4.65 GHz

Hold time constraint:

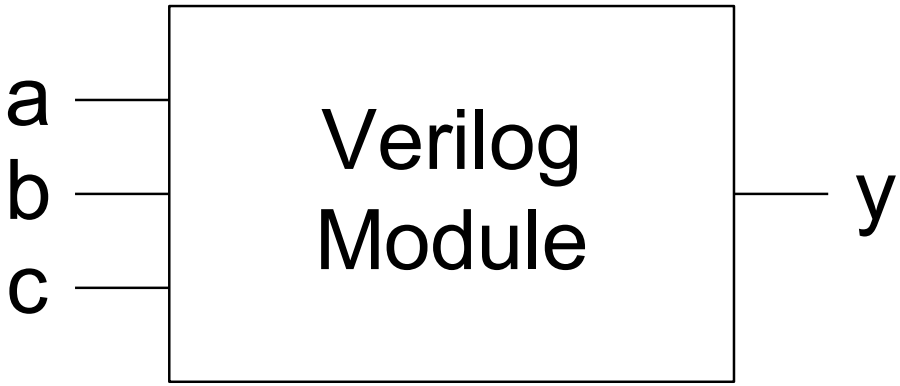
$t_{ccq} + t_{cd} > t_{hold}$?

(30 + 25) ps > 70 ps ? **No!**

© *Digital Design and Computer Architecture*, 2nd Edition, 2012

ELSEVIER

Verilog Modules



Two types of Modules:

- Behavioral: describe what a module does
- Structural: describe how a module is built from simpler modules

Copyright © 2007 Elsevier

Behavioral Verilog Example

Verilog:

```
module example(input  a, b, c,
               output y);
  assign y = ~a & ~b & ~c | a & ~b & ~c | a & ~b &  c;
endmodule
```

Copyright © 2007 Elsevier

Structural Modeling - Hierarchy

```
module and3(input  a, b, c,
            output y);
  assign y = a & b & c;
endmodule


module inv(input  a,
           output y);
  assign y = ~a;
endmodule


module nand3(input  a, b, c
             output y);
  wire n1;                      // internal signal

  and3 andgate(a, b, c, n1);  // instance of and3
  inv  inverter(n1, y);      // instance of inverter
endmodule
```

Copyright © 2007 Elsevier

Basic Operations and Order

Defines the order of operations

Highest

this tripped me up, during lab.

~	NOT
*, /, %	mult, div, mod
+, -	add,sub
<<, >>	shift
<<<, >>>	arithmetic shift
<, <=, >, >=	comparison
==, !=	equal, not equal
&, ~&	AND, NAND
^, ~^	XOR, XNOR
|, ~|	OR, NOR
?:	ternary operator

Lowest

Copyright © 2007 Elsevier

ELSEVIER

Numbers

Format: N'Bvalue

N = number of bits, B = base

N'B is optional but recommended (default is decimal)

Number	# Bits	Base	Decimal Equivalent	Stored
3’b101	3	binary	5	101
‘b11	unsized	binary	3	00…0011
8’b11	8	binary	3	00000011
8’b1010_1011	8	binary	171	10101011
3’d6	3	decimal	6	110
6’o42	6	octal	34	100010
8’hAB	8	hexadecimal	171	10101011
42	Unsized	decimal	42	00…0101010

Copyright © 2007 Elsevier

Bit Manipulations: Example 1

```
assign y = {a[2:1], {3{b[0]}}, a[0], 6'b100_010};

// if y is a 12-bit signal, the above statement produces:
y = a[2] a[1] b[0] b[0] b[0] a[0] 1 0 0 0 1 0

// underscores (_) are used for formatting only to make
   it easier to read. Verilog ignores them.
```

Copyright © 2007 Elsevier

Bit Manipulations: Example 2

Verilog:

```
module mux2_8(input  [7:0] d0, d1,
              input        s,
              output [7:0] y);

  mux2 lsbmux(d0[3:0], d1[3:0], s, y[3:0]);
  mux2 msbmux(d0[7:4], d1[7:4], s, y[7:4]);
endmodule
```

Copyright © 2007 Elsevier

Always Statement

General Structure:

```
always @ (sensitivity list)
   statement;
```

Whenever the event in the `sensitivity list` occurs, the `statement` is executed

Copyright © 2007 Elsevier

Combinational Logic using **always**

```
// combinational logic using an always statement
module gates(input       [3:0] a, b,
             output reg [3:0] y1, y2, y3, y4, y5);
  always @(*)        // need begin/end because there is
    begin             // more than one statement in always
      y1 = a & b;     // AND
      y2 = a | b;     // OR
      y3 = a ^ b;     // XOR
      y4 = ~(a & b); // NAND
      y5 = ~(a | b); // NOR
    end
endmodule
```

This hardware could be described with assign statements using fewer lines of code, so it's better to use assign statements in this case.

Copyright © 2007 Elsevier

Combinational Logic using case

```
module sevenseg(input       [3:0] data,
                output reg [6:0] segments);
  always @(*)
    case (data)
      //                abc_defg
      0: segments = 7'b111_1110;
      1: segments = 7'b011_0000;
      2: segments = 7'b110_1101;
      3: segments = 7'b111_1001;
      4: segments = 7'b011_0011;
      5: segments = 7'b101_1011;
      6: segments = 7'b101_1111;
      7: segments = 7'b111_0000;
      8: segments = 7'b111_1111;
      9: segments = 7'b111_1011;
      default: segments = 7'b000_0000; // required
    endcase
endmodule
```

Copyright © 2007 Elsevier

Rules for Signal Assignment

- Use `always @(posedge clk)` and nonblocking assignments (<=) to model synchronous sequential logic

```
always @ (posedge clk)
  q <= d; // nonblocking
```

- Use continuous assignments (assign …)to model simple combinational logic.

```
assign y = a & b;
```

- Use `always @ (*)` and blocking assignments (=) to model more complicated combinational logic where the `always` statement is helpful.

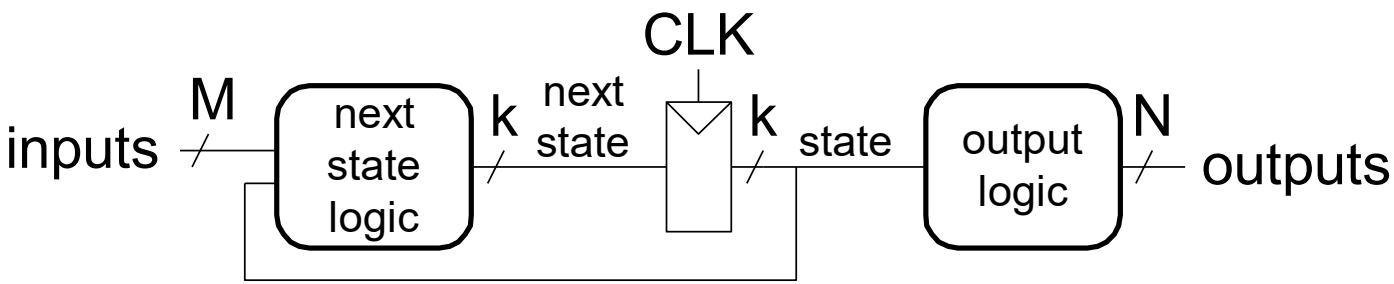
- Do not make assignments to the same signal in more than one `always` statement or continuous assignment statement.

Copyright © 2007 Elsevier

ELSEVIER

Finite State Machines (FSMs)

- Three blocks:
  - next state logic
  - state register
  - output logic



Copyright © 2007 Elsevier

FSM in Verilog

```
module divideby3FSM (input  clk,
                     input  reset,
                     output q);
   reg  [1:0] state, nextstate;

   parameter S0 = 2'b00;
   parameter S1 = 2'b01;
   parameter S2 = 2'b10;

// state register
   always @ (posedge clk, posedge reset)
      if (reset) state <= S0;
      else       state <= nextstate;
 // next state logic
   always @ (*)
      case (state)
         S0:      nextstate = S1;
         S1:      nextstate = S2;
         S2:      nextstate = S0;
         default: nextstate = S0;
      endcase
   // output logic
   assign q = (state == S0);
endmodule
```

I should have a clear mental model of how to build verilog code.

Copyright © 2007 Elsevier